EMBEDDED LINUX SYSTEMS

ASSIGNMENT 01
HOW TO WRITE A DEVICE DRIVER

Chủ đề: ioctl Interface | Misc Device | Driver Analysis | ARMv7

Sinh viên: Nguyễn Trần Nam Nguyên

Mã số sinh viên: SE205174

Nhóm: 10

Môi trường: Ubuntu, Linux 5.15, ARMv7, QEMU vexpress-a9

BẢN CẬP NHẬT BÁO CÁO KỸ THUẬT - CẦN SINH VIÊN RÀ SOÁT VÀ TỰ VẼ LẠI SƠ ĐỒ TRƯỚC KHI NỘP

1. Mục tiêu và phạm vi

Assignment 01 yêu cầu mở rộng character driver từ LAB-02 bằng giao diện ioctl và xây dựng một misc device driver độc lập. Báo cáo tập trung vào cơ chế giao tiếp userspace–kernelspace, quyết định thiết kế, tính đúng đắn của các thao tác và bằng chứng kiểm thử trên ARMv7.

1.1. Mục tiêu kỹ thuật

• Task 1: cung cấp bốn lệnh ioctl để xóa bộ đệm, đọc thống kê, chọn chế độ ghi và lấy phiên bản driver.

• Task 2: xây dựng thiết bị /dev/counter, dùng atomic_t, hỗ trợ đọc tăng tự động, ghi đặt giá trị và công bố thống kê qua procfs.

• Task 3: phân tích kiến trúc, so sánh ba nhóm driver và đánh giá quyết định thiết kế.

• Task 4: giải thích reference count, bảo vệ vùng nhớ userspace, MTD và file offset.

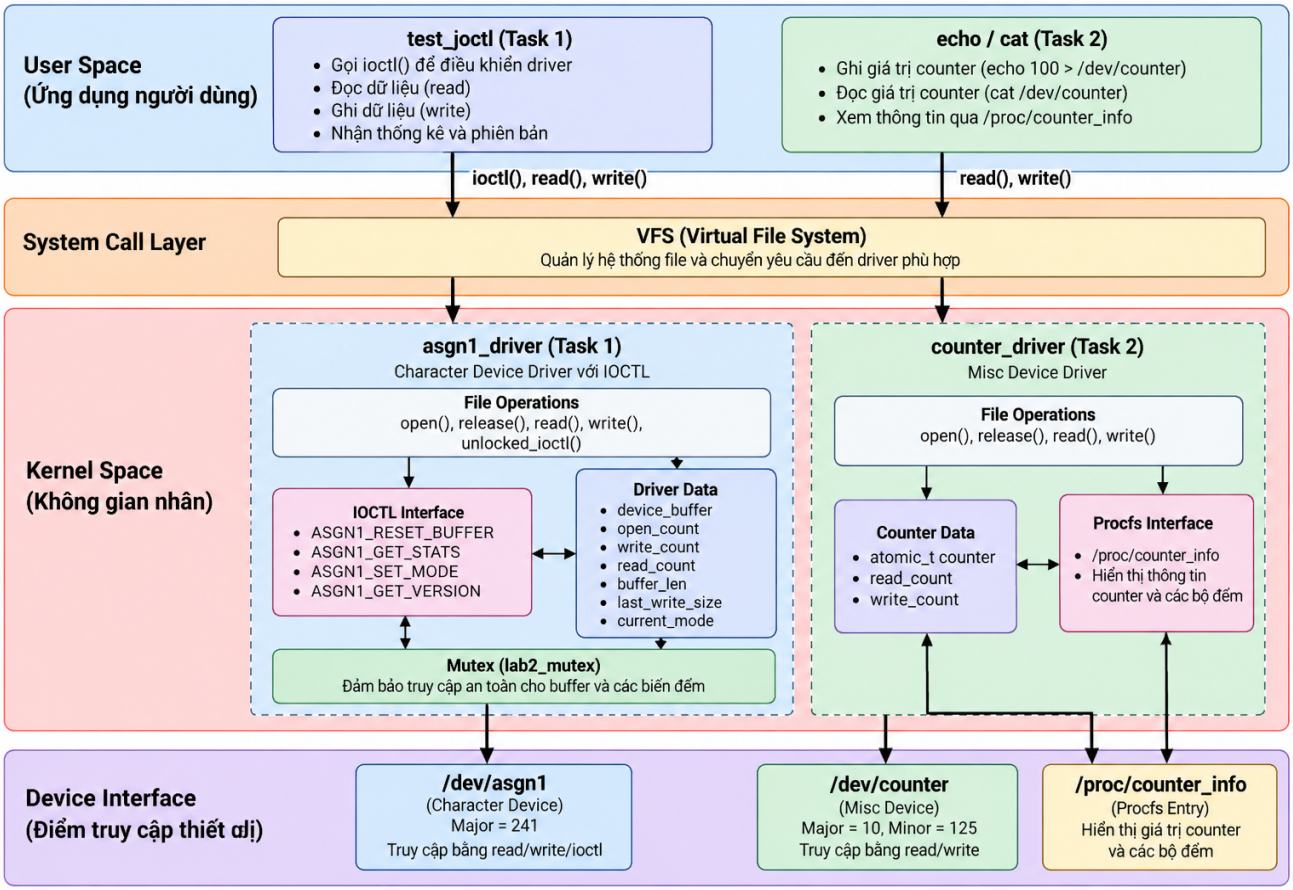
1.2. Môi trường thực nghiệm

Thành phần	Cấu hình đã sử dụng
Máy phát triển	Ubuntu; cross-compiler arm-linux-gnueabihf-
Kernel đích	Linux 5.15, kiến trúc ARMv7-A
Máy mô phỏng	QEMU vexpress-a9, RAM 256 MB
Hệ thống tối thiểu	BusyBox và initramfs
Driver	asgn1_driver.ko; counter_driver.ko

Các nhận xét dựa trên ảnh chụp kiểm thử, kết quả QEMU, nội dung asgn1_ioctl.h và trích đoạn hàm asgn1_ioctl() do sinh viên cung cấp. Phần còn thiếu chứng cứ được nêu rõ để sinh viên kiểm tra.

2. Kiến trúc giao tiếp driver

Sơ đồ kiến trúc minh họa hai nhánh Task 1 và Task 2. Đây là hình minh họa do công cụ hỗ trợ tạo, không phải sơ đồ tự vẽ của sinh viên; cần tự dựng lại và rà soát các chi tiết trước khi nộp theo quy định đề bài.



Hình 0. Sơ đồ minh họa kiến trúc hai driver (tham khảo; cần tự vẽ lại). Một số ô file_operations là mô tả khái niệm, không phải khai báo chính xác từng callback trong mã nguồn.

Luồng ioctl Task 1: ứng dụng gọi ioctl(fd, command, argument) → VFS điều phối đến unlocked_ioctl → driver xác thực command và dữ liệu → thao tác bộ đệm/trạng thái → copy_to_user hoặc copy_from_user khi cần → trả mã lỗi/kết quả về userspace.

Task 2 không triển khai ioctl. Các thao tác đọc/ghi đi qua miscdevice và file_operations; thông tin thống kê được xuất qua procfs, là giao diện riêng để quan sát trạng thái driver.

3. Task 1 - Thiết kế IOCTL Driver

Driver asgn1_driver mở rộng mô hình character device từ LAB-02. Thiết bị được kiểm thử qua /dev/asgn1; nhật ký ghi nhận major number 241. Giao diện ioctl tách các thao tác điều khiển khỏi đường truyền dữ liệu read/write.

Lệnh ioctl	Ý nghĩa và kết quả mong đợi
ASGN1_RESET_BUFFER	Xóa dữ liệu đang lưu; đặt read_count và write_count về 0.
ASGN1_GET_STATS	Trả struct thống kê: open_count, write_count, read_count, buffer_len, last_write_size.
ASGN1_SET_MODE	Mode 0 thêm tiền tố [KERNEL]; mode 1 giữ nguyên dữ liệu.
ASGN1_GET_VERSION	Trả chuỗi phiên bản với giới hạn vùng đệm userspace 64 byte.

3.1. Điều khiển và an toàn dữ liệu

Đối chiếu file asgn1_ioctl.h thực tế: ASGN1_IOC_MAGIC = ký tự A; RESET_BUFFER = _IO(A, 0); GET_STATS = _IOR(A, 1, struct asgn1_stats); SET_MODE = _IOW(A, 2, int); GET_VERSION = _IOR(A, 3, char[64]). Cấu trúc asgn1_stats gồm open_count, write_count, read_count, buffer_len và last_write_size. _IOC_NR(GET_STATS) = 1; _IOC_SIZE(GET_STATS) = sizeof(struct asgn1_stats), tương ứng 20 byte với năm trường int 32-bit trên ARMv7.

Trong hàm asgn1_ioctl(), RESET_BUFFER khóa lab2_mutex trước khi đặt buffer_len, read_count, write_count và last_write_size về 0; GET_STATS lấy snapshot năm trường trong mutex rồi copy_to_user(); SET_MODE dùng copy_from_user(), chỉ nhận 0 hoặc 1 rồi cập nhật current_mode dưới mutex; GET_VERSION sao chép mảng version 64 byte; command không hỗ trợ trả -ENOTTY. Mã nguồn vẫn còn tên lab2_mutex, lab2_fops và một số log lab2_driver do kế thừa LAB02.

3.2. Hành vi đã quan sát

Ảnh kiểm thử ghi nhận phiên bản “asgn1_driver v1.0 - SE205174”, mode 0 cho kết quả “[KERNEL]Hello”, reset làm các bộ đếm đọc/ghi và chiều dài buffer về 0; mode 1 trả “Hello Echo”. Chế độ không hợp lệ và command ioctl không hỗ trợ đều bị từ chối.

4. Task 1 - Kết quả kiểm thử

Hình 1 là ảnh đầu ra kiểm thử Task 1 được cung cấp. Ảnh thể hiện các lệnh ioctl, các giá trị thống kê và các thông báo kernel; cuối ảnh có dấu mốc “TASK 1 TEST COMPLETED”.



Hình 1. Kết quả kiểm thử Task 1 (ảnh do sinh viên cung cấp).

4.1. Nhận xét

• GET_VERSION và GET_STATS trả về thông tin hợp lệ; thống kê ban đầu có open_count = 1.

• SET_MODE 0/1 tạo đúng dữ liệu “[KERNEL]Hello” và “Hello Echo”.

• RESET_BUFFER báo thành công và thống kê sau reset có write_count = 0, read_count = 0, buffer_len = 0.

• Kiểm tra mode không hợp lệ và ioctl không hỗ trợ đều thành công.

4.2. Điểm cần rà soát

Trong ảnh, một số thông báo kernel vẫn mang tiền tố “lab2_driver” thay vì “asgn1_driver”. Đây là dấu vết từ LAB-02, không phải bằng chứng lỗi chức năng, nhưng nên đổi nhãn log trong mã nguồn, biên dịch lại và kiểm thử lại để thống nhất báo cáo. Ảnh chưa cho phép xác nhận đầy đủ mọi đường mutex hoặc kích thước copy_to_user; cần đối chiếu source.

5. Task 2 - Thiết kế Misc Counter Driver

Counter driver là module độc lập với Task 1, đăng ký bằng misc_register() với major number 10 và minor động. Trong môi trường initramfs thử nghiệm, device node /dev/counter được tạo thủ công sau khi tra cứu /proc/misc.

5.1. Hoạt động của read và write

Hàm read lấy giá trị counter hiện tại, tăng biến đếm bằng thao tác atomic và trả chuỗi thập phân có newline. Điều kiện *ppos > 0 giúp kết thúc lượt đọc sau khi đã cung cấp dữ liệu. Hàm write sao chép chuỗi từ userspace, chuyển sang số nguyên bằng kstrtoint(), từ chối giá trị âm và cập nhật counter bằng atomic_set().

5.2. Quan sát thống kê

File /proc/counter_info hiển thị Current counter, Read count và Write count. Giao diện này được tạo bằng proc_create() và seq_file, đồng thời được giải phóng bằng proc_remove() khi module bị gỡ.

5.3. Lý do dùng atomic_t

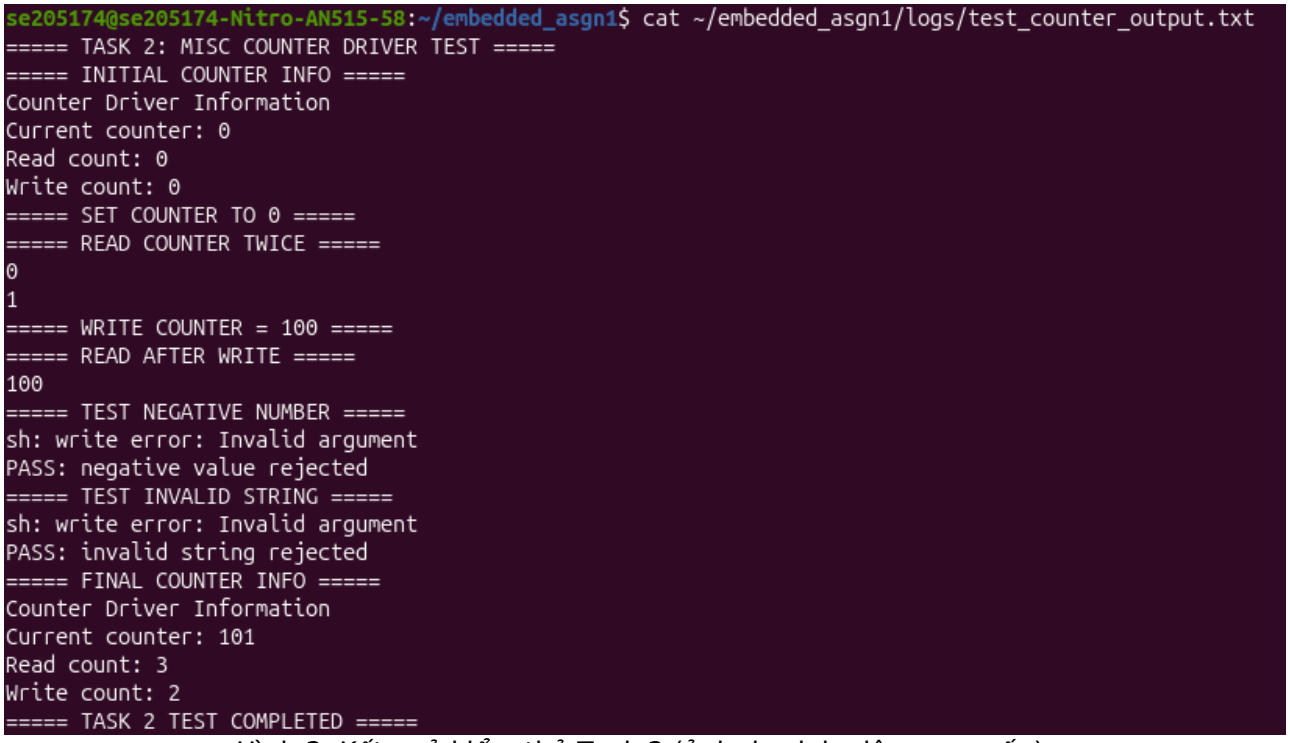
Giả sử hai tiến trình cùng đọc giá trị counter = 5. Nếu dùng int và thao tác counter++ không đồng bộ, cả hai có thể cùng đọc 5 rồi cùng ghi 6, khiến một lần tăng bị mất. Thao tác atomic bảo đảm các phép tăng độc lập không bị mất cập nhật theo cách đó. Tuy nhiên, ba số liệu xuất ra procfs được đọc riêng rẽ nên không tạo thành một ảnh chụp trạng thái tuyệt đối đồng thời.

5.4. Xử lý lỗi và vòng đời

Driver trả -EINVAL cho số âm hoặc chuỗi không chuyển được sang số. Khi khởi tạo, nếu tạo procfs thất bại sau khi đăng ký miscdevice, cần hủy đăng ký thiết bị trước khi trả lỗi. Khi unload, phải xóa procfs và deregister miscdevice. Kiểm thử thực tế đã xác nhận các tài nguyên đăng ký không còn trong /proc sau rmmod.

6. Task 2 - Kết quả kiểm thử

Hình 2 cho thấy đầu ra tự động của test_counter.sh. Bài test đã ghi 0, đọc hai lần, ghi 100, đọc tiếp và kiểm tra hai trường hợp dữ liệu không hợp lệ.



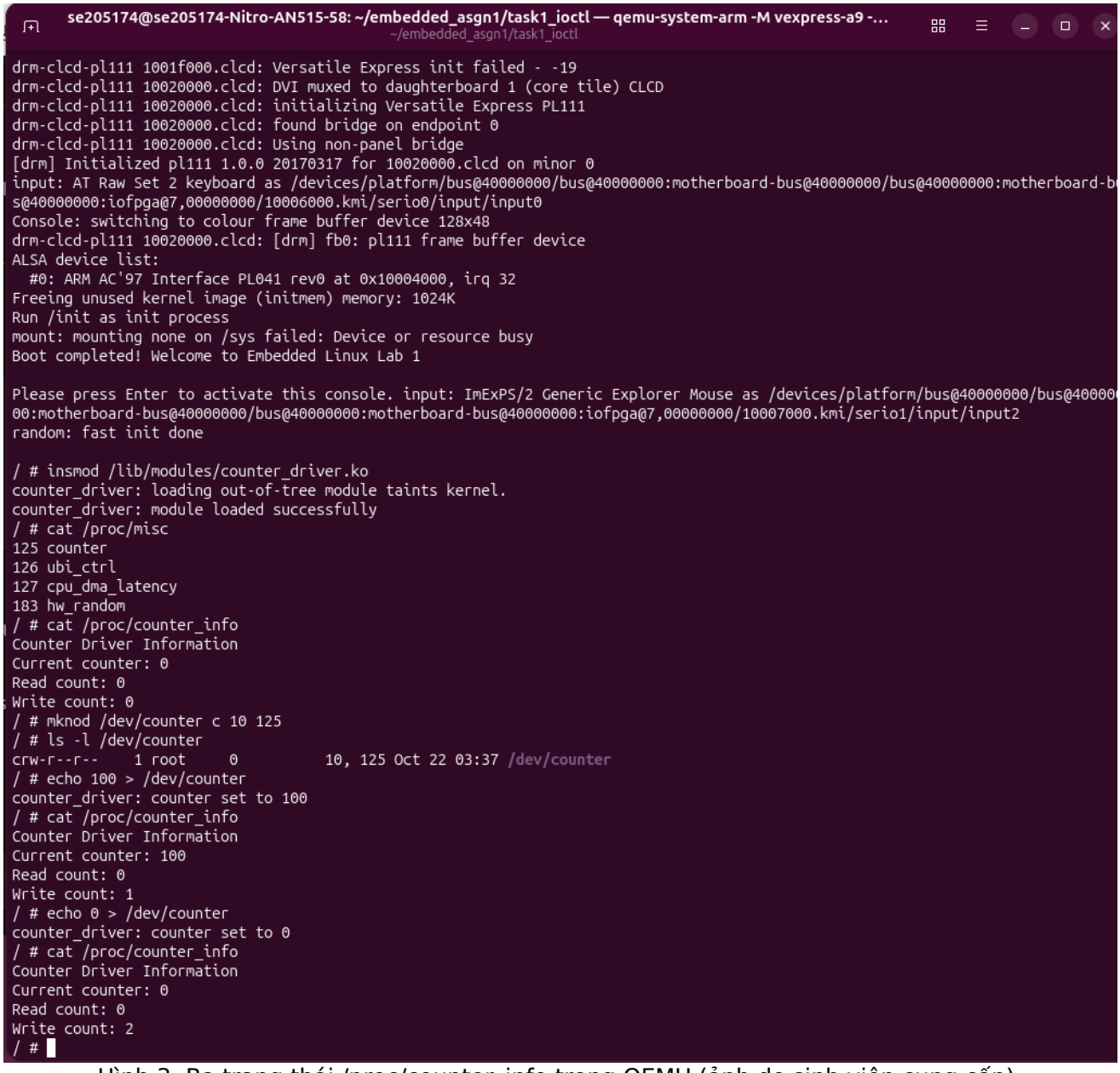
Hình 2. Kết quả kiểm thử Task 2 (ảnh do sinh viên cung cấp).

Thao tác	Kết quả quan sát
Đọc sau khi ghi 0	Lần 1: 0; lần 2: 1
Ghi 100, đọc tiếp	Trả về 100
Ghi -5 / abc	Invalid argument; hai bài kiểm tra PASS
Trạng thái cuối	Current counter = 101; Read count = 3; Write count = 2

Ảnh Task 2 phía trên chứng minh các phép read/write và xử lý -EINVAL. Ảnh trạng thái trước/sau ghi và sau đặt lại về 0, cùng các dòng dmesg, được bổ sung ở trang kế tiếp.

6.1. Minh chứng bổ sung: Counter và dmesg

Ảnh kiểm thử QEMU bổ sung cho thấy ba thời điểm của /proc/counter_info trong cùng một phiên nạp module: trạng thái ban đầu, sau echo 100 và sau echo 0.



Hình 3. Ba trạng thái /proc/counter_info trong QEMU (ảnh do sinh viên cung cấp).

Thời điểm	Current counter	Read count	Write count
Ban đầu	0	0	0
Sau echo 100	100	0	1
Sau echo 0	0	0	2

Ở đây reset có nghĩa là ghi giá trị 0 vào counter, không phải đặt lại read_count hoặc write_count. Hai lần ghi hợp lệ làm write_count tăng từ 0 lên 2.

6.2. Phân tích kernel log của hai driver

Task 1: asgn1_driver (dmesg đã lưu)

Log ghi nhận major=241, GET_VERSION called, GET_STATS called, mode changed to 0, received 5 bytes, sent 13 bytes to user, buffer and counters reset, mode changed to 1, received 10 bytes, sent 10 bytes to user và module unloaded. Dòng gửi 13 byte phù hợp với [KERNEL]Hello (8 ký tự tiền tố + 5 ký tự Hello).

Trong log có cả tiền tố lab2_driver và asgn1_driver do mã nguồn Task 1 phát triển từ LAB02. Một số chuỗi xuất hiện lặp lại vì đã chạy hai lượt kiểm thử.

Task 2: counter_driver (dmesg từ QEMU)

Dòng log quan sát	Ý nghĩa
counter_driver: module loaded successfully	misc_register và proc_create thành công
counter_driver: counter set to 100	write hợp lệ đặt counter thành 100
counter_driver: counter set to 0	write hợp lệ đặt counter về 0

Dòng loading out-of-tree module taints kernel chỉ ra module ngoài cây mã nguồn kernel đã được nạp; riêng thông báo này không chứng minh lỗi thực thi. Kiểm thử unload trước đó cho thấy mục counter biến mất khỏi /proc/misc và /proc/counter_info được giải phóng.

Nguồn minh chứng: logs/dmesg_full.txt (Task 1), ảnh/đầu ra QEMU (Task 2), Task1.png, Task2.png và ảnh kiểm thử ba thời điểm.

7. So sánh ba loại Linux Device Driver

Tiêu chí	Character	Block	Network
Giao diện chính	file_operations	block_device_operations	net_device_ops
Đơn vị dữ liệu	Luồng byte	Khối/sector	Gói/frame
Buffering	Tùy driver; buffer nội bộ hoặc kernel	Page cache, I/O queue, block layer	Socket buffer, hàng đợi TX/RX, NAPI
Ví dụ nhúng	UART, cảm biến, /dev/counter	eMMC, thẻ SD	Ethernet MAC, Wi-Fi
Phù hợp khi	Điều khiển tuần tự, dữ liệu nhỏ	Lưu trữ truy cập ngẫu nhiên	Trao đổi gói qua mạng

7.1. Hai quyết định thiết kế cụ thể

Ví dụ 1: thiết bị cảm biến chỉ trả một giá trị và có lệnh điều khiển riêng nên dùng character/misc device; không cần tầng block và không biểu diễn thành packet mạng.

Ví dụ 2: lưu hệ thống tập tin trên eMMC/SD nên đi qua block layer để tận dụng hàng đợi I/O và cache. Trái lại, giao tiếp Ethernet cần network device vì phải xử lý frame, địa chỉ MAC và giao thức mạng.

7.2. register_chrdev và misc_register

Task 1 sử dụng cơ chế đăng ký character device trực tiếp để quản lý major và các thao tác riêng, phù hợp khi cần kiểm soát tên/thiết bị và phát triển từ LAB-02. Task 2 dùng misc_register() giúp giảm mã đăng ký cho thiết bị nhỏ, dùng chung major 10 và nhận minor riêng. Đổi lại, misc framework ít linh hoạt hơn khi cần một mô hình thiết bị phức tạp với nhiều minor và quy tắc quản lý riêng.

8. ioctl encoding và phân tích lỗi

8.1. Magic number và trường mã hóa

Macro ioctl mã hóa loại lệnh (type/magic), số thứ tự (number), hướng truyền (direction) và kích thước dữ liệu (size). Magic number giúp phân biệt nhóm command; _IOC_NR(cmd) trích số thứ tự và _IOC_SIZE(cmd) trích kích thước đã mã hóa. Với _IOR, dữ liệu được trả từ kernel sang userspace; _IOW truyền từ userspace vào kernel; _IO không có payload.

Ví dụ đối chiếu header thực tế: ASGN1_GET_STATS = _IOR(ASGN1_IOC_MAGIC, 1, struct asgn1_stats), với ASGN1_IOC_MAGIC = A. Vì vậy _IOC_NR(ASGN1_GET_STATS) = 1; _IOC_SIZE(ASGN1_GET_STATS) = sizeof(struct asgn1_stats) = 20 byte trên ARMv7 với năm int 32-bit. _IOC_TYPE trả về mã của A; _IOC_DIR thể hiện hướng kernel sang userspace.

8.2. Debugging trong quá trình thực hiện

Sự cố thực tế 1: QEMU không tự xuất hiện /dev/counter dù misc_register thành công. Kiểm tra /proc/misc cho thấy minor 125; node ban đầu tạo nhầm minor 123. Cách khắc phục: xóa node sai và tạo lại “mknod /dev/counter c 10 125”. Sau đó read/write hoạt động đúng.

Sự cố thực tế 2: khi chuyển log bằng nc, ping 10.0.2.2 báo Network is unreachable dù DHCP đã thông báo cấp lease. Lệnh ip addr show eth0 chưa có địa chỉ inet. Gán thủ công 10.0.2.15/24 cho eth0, kiểm tra ping và truyền file thành công.

Giới hạn xác minh: đã đối chiếu header và đoạn hàm ioctl do sinh viên gửi, nhưng chưa rà soát toàn bộ các đường lỗi, concurrency và cleanup trong cả hai file nguồn. Log Task 2 được cung cấp dưới dạng đầu ra QEMU, chưa xác nhận đã gộp vào dmesg_full.txt trên Ubuntu.

9. Task 4 - Câu hỏi lý thuyết

Câu 1. Module reference counting

Khi file của driver được mở, nếu file_operations.owner = THIS_MODULE, VFS giữ tham chiếu module; khi đóng file, tham chiếu được nhả. Các đường sử dụng module_get()/module_put() có thể tăng/giảm tham chiếu tường minh. Nếu còn tham chiếu, rmmod thông thường thất bại với trạng thái module đang được sử dụng. Force unload chỉ có thể có trên kernel được cấu hình hỗ trợ và rất nguy hiểm (có thể gây crash hoặc use-after-free), nên cách đúng là đóng các file đang mở và dừng tiến trình sử dụng trước khi unload.

Câu 2. Tại sao dùng copy_to_user()?

Con trỏ userspace là địa chỉ ảo thuộc không gian địa chỉ của tiến trình, không phải vùng nhớ kernel có thể tùy ý ghi bằng memcpy. MMU phân giải địa chỉ qua page table và áp dụng quyền truy cập; trang có thể chưa được ánh xạ, bị swap hoặc không có quyền ghi. Truy cập trực tiếp có thể gây page fault không được xử lý đúng ngữ cảnh, lỗi bảo vệ bộ nhớ hoặc lỗi kernel. copy_to_user() dùng cơ chế truy cập bộ nhớ người dùng có kiểm tra và báo số byte chưa sao chép; driver cần xử lý thất bại, thường trả -EFAULT.

Câu 3. MTD và block subsystem

MTD phục vụ thiết bị nhớ flash thô, nơi phần mềm phải quan tâm xóa theo eraseblock, bad block, wear và đặc tính ghi/xóa. Block layer cung cấp giao diện sector, phù hợp eMMC/SD/SSD có bộ điều khiển che giấu các đặc tính flash. Ví dụ NAND flash thô gắn trực tiếp với SoC thường dùng MTD kết hợp UBI/UBIFS; eMMC thông thường dùng block device.

Câu 4. Ý nghĩa *ppos trong read()

Lệnh cat gọi read() lặp lại cho đến khi read trả 0 (EOF) hoặc lỗi. Nếu mỗi lần read của /dev/counter luôn trả một chuỗi khác rỗng mà không xét *ppos, cat sẽ tiếp tục đọc và in số không ngừng. Kiểm tra *ppos > 0 để trả 0 ở lần đọc tiếp theo giúp mỗi lần mở file chỉ trả một giá trị. Sau khi sao chép thành công, driver tăng *ppos theo số byte trả về.

10. Đánh giá, bổ sung và kết luận

10.1. Đánh giá thực nghiệm

Kết quả đã quan sát xác nhận hai driver chạy trên QEMU ARMv7, xử lý các thao tác chính, từ chối đầu vào không hợp lệ và hoàn thành script kiểm thử. Việc gỡ counter_driver còn được xác nhận bằng thông báo module unloaded, lsmod rỗng, /proc/counter_info biến mất và mục counter không còn trong /proc/misc.

10.2. Những việc cần hoàn thiện trước khi nộp

• Tự vẽ lại sơ đồ kiến trúc tham khảo theo yêu cầu học thuật của đề bài; kiểm tra lại các callback file_operations trước khi đưa vào bản chính thức.

• Đã đối chiếu magic A, NR 0–3 và size GET_STATS = 20 byte với header; rà soát lại mã nguồn toàn phần trước khi nộp.

• Đã bổ sung ba trạng thái /proc/counter_info. Nếu đề bài yêu cầu thêm tình huống đọc xen kẽ, cần kiểm thử và ghi nhận riêng.

• Đã có log dmesg Task 1 và đầu ra dmesg Task 2; cần lưu thêm log Task 2 vào file tổng hợp trên Ubuntu nếu yêu cầu nộp bản log chung.

• Rà soát nhãn “lab2_driver” còn sót trong log Task 1 và xác nhận khóa mutex trong source.

• Rà soát nội dung Task 4 bằng cách tự diễn đạt lại theo hiểu biết và trích dẫn tài liệu đã dùng.

10.3. Kết luận

Bài thực hành cho thấy sự khác biệt giữa một character driver có ioctl và một misc device đơn mục đích. ioctl giúp mở rộng giao diện điều khiển có cấu trúc; miscdevice rút gọn việc đăng ký thiết bị; atomic_t hỗ trợ cập nhật bộ đếm an toàn trong môi trường đồng thời. Kết quả thử nghiệm là nền tảng để tiếp tục hoàn thiện thiết kế và tài liệu kỹ thuật.

Tài liệu tham khảo

1. Tài liệu “ASSIGNMENT-01 | Embedded Linux Systems | How to Write a Device Driver” do giảng viên cung cấp.

2. Linux Kernel Documentation, driver-api/: kernel.org/doc/html/latest/driver-api/

3. Corbet, Rubini, Kroah-Hartman, Linux Device Drivers (3rd Edition), LWN.net.

4. Linux man-pages: ioctl(2), open(2), read(2), write(2).